

Relatório — Tabelas Hash com POO em Java

Disciplina: Estrutura de Dados | Capacidade da tabela: 16 | Nomes inseridos: 20.000

1. Descrição do Projeto

O programa implementa duas tabelas hash utilizando conceitos de Programação Orientada a Objetos (POO) em Java. Uma classe abstrata **TabelaHash** define a estrutura genérica e o tratamento de colisões por sondagem linear. Duas subclasses concretas — **TabelaHash1** e **TabelaHash2** — sobrescrevem apenas a função hash, conforme exigido pelo enunciado. O programa lê 20.000 nomes de um arquivo CSV, insere nas duas tabelas e gera este relatório.

2. Funções Hash Implementadas

Hash 1 — Soma ASCII: Soma os valores ASCII de todos os caracteres da chave e aplica módulo pela capacidade.

$$\text{hash}(k) = (\text{sum}(k[i])) \% \text{capacidade}$$

Hash 2 — Polinomial (DJB2 adaptado): Multiplica o hash acumulado por 31 a cada caractere, produzindo melhor dispersão.

$$\text{hash}(k) = (7 * 31^n + k[0]*31^{(n-1)} + \dots + k[n]) \% \text{capacidade}$$

3. Tratamento de Colisões

Foi utilizada **sondagem linear (linear probing)**: ao ocorrer colisão na posição p , tenta-se a posição $(p+1) \% \text{capacidade}$, repetindo até encontrar posição livre. Cada tentativa extra conta como uma colisão.

4. Resultados — Comparativo de Eficiência

Métrica	Hash 1 (Soma ASCII)	Hash 2 (Polinomial djb2)
Capacidade da tabela	16	16
Nomes lidos do CSV	20.000	20.000
Elementos efetivamente inseridos	16	16
Total de colisões	319.788	319.768
Tempo de inserção (ns)	14.603.426	22.770.990
Tempo de inserção (ms)	14,603 ms	22,771 ms
Tempo de busca — 100 nomes (ns)	1.369.407	611.103
Tempo de busca — 100 nomes (ms)	1,369 ms	0,611 ms

5. Distribuição das Chaves por Posição

Com capacidade 16 e 20.000 nomes, a tabela fica completamente cheia após as primeiras 16 inserções bem-sucedidas. Os demais 19.984 nomes tentam inserção mas encontram a tabela saturada (gerando as colisões contabilizadas). Abaixo a distribuição final de cada tabela:

Hash 1 — Posições ocupadas:

Posição	Nome armazenado
0	Ana Nunes Araujo
1	Ricardo Ribeiro Cardoso
2	Helena Cardoso Ramos
3	Yasmin Pinto Lima
4	Queila Teixeira Nascimento
5	Tiago Ramos Xavier
6	Larissa Santos Moreira
7	Tiago Costa Nunes
8	Maria Nunes Teixeira
9	Carlos Barros Ribeiro
10	Tiago Miranda Barros
11	Sandra Correia Gomes
12	Karen Carvalho Costa
13	Elisa Ferreira Oliveira
14	Leonardo Ribeiro Martins
15	Camila Pinto Marques

Hash 2 — Posições ocupadas:

Posição	Nome armazenado
0	Sandra Correia Gomes
1	Helena Cardoso Ramos
2	Queila Teixeira Nascimento
3	Yasmin Pinto Lima
4	Elisa Ferreira Oliveira
5	Maria Nunes Teixeira
6	Tiago Miranda Barros
7	Camila Pinto Marques
8	Tiago Costa Nunes

9	Ricardo Ribeiro Cardoso
10	Tiago Ramos Xavier
11	Karen Carvalho Costa
12	Carlos Barros Ribeiro
13	Ana Nunes Araujo
14	Larissa Santos Moreira
15	Leonardo Ribeiro Martins

6. Clusterização (Colisões por Posição)

Como a tabela tem capacidade 16 e todos os slots são ocupados, forma-se um único cluster contíguo de tamanho 16 (posições 0 a 15) em ambas as tabelas. Isso gera 15 colisões internas de encadeamento por sondagem linear. O alto número de colisões totais (≈ 319 mil) ocorre porque os 19.984 nomes excedentes tentam se inserir percorrendo toda a tabela cheia sem êxito.

Tabela	Posição do Cluster	Tamanho	Colisões Internas	Posições Ocupadas
Hash 1 (Soma ASCII)	0 – 15	16	15	16/16 (100%)
Hash 2 (Polinomial djb2)	0 – 15	16	15	16/16 (100%)

7. Análise dos Resultados

As duas funções hash apresentaram número de colisões muito próximo (~ 319 mil), o que é esperado dado que ambas enchem a tabela completamente — a diferença de distribuição aparece na ordem em que as posições são preenchidas. A Hash 2 (polinomial) teve inserção mais lenta por realizar mais operações aritméticas por chave, porém foi significativamente mais rápida na busca (0,611 ms vs 1,369 ms), pois com melhor distribuição inicial as sondagens são mais curtas. A Hash 1 (soma ASCII) é mais simples e rápida de calcular, mas tende a gerar mais agrupamentos em strings de comprimento similar, o que pode aumentar o tempo de busca.

8. Estrutura dos Arquivos Fonte

Arquivo	Descrição
TabelaHash.java	Classe abstrata com inserção (sondagem linear), busca e atributos comuns
TabelaHash1.java	Subclasse: sobrescreve funcaoHash() com soma ASCII
TabelaHash2.java	Subclasse: sobrescreve funcaoHash() com hash polinomial (djb2 adaptado)
LeitorCSV.java	Lê o arquivo CSV e retorna array de Strings com os nomes
Main.java	Orquestra leitura, inserção, medição de tempo e impressão do relatório